A Appendix

A.1 Structure of the proposed ASFF method.

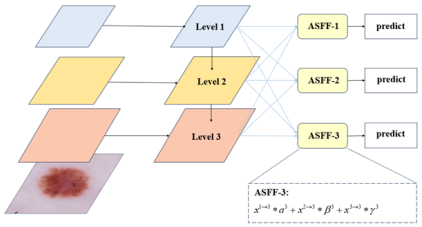


Fig. 6: Structure of the proposed ASFF method.

A.2 ASFF

ASFF's key idea is to adaptively learn the spatial weight of fusion for feature maps at each scale. It consists of two steps: identically rescaling and adaptively fusing.

Feature Resizing. We denote the features of the resolution at level l ($l \in \{1, 2, 3\}$ for YOLOv3) as $\mathbf{x}^l$. For level l, we resize the features $\mathbf{x}^n$ at the other level n ($n \neq l$) to the same shape as that of $\mathbf{x}^l$. Because the features at three levels in YOLOv3 have different resolutions as well as different numbers of channels, we accordingly modify the up-sampling and down-sampling strategies for each scale. For up-sampling, we first apply a 1×1 convolution layer to compress the number of channels of features to that in level l, and then upscale the resolutions respectively with interpolation. For down-sampling with $1/2$ ratio, we simply use a 3×3 convolution layer with a stride of 2 to modify the number of channels and the resolution simultaneously. For the scale ratio of $1/4$, we add a 2-stride max pooling layer before the 2-stride convolution.

Adaptive Fusion. Let $\mathbf{x}_{ij}^{n \to l}$ denote the feature vector at the position (i, j) on the feature maps resized from level n to level l. We propose to fuse the features at the corresponding level l as follows:

$$\mathbf{y}_{ij}^l = \alpha_{ij}^l \cdot \mathbf{x}_{ij}^{1 \to l} + \beta_{ij}^l \cdot \mathbf{x}_{ij}^{2 \to l} + \gamma_{ij}^l \cdot \mathbf{x}_{ij}^{3 \to l}, \tag{4}$$

where $\mathbf{y}_{ij}^l$ implies the (i, j)-th vector of the output feature maps $\mathbf{y}^l$ among channels. α_{ij}^l, β_{ij}^l and γ_{ij}^l refer to the spatial importance weights for the feature maps at three different levels to level l, which are adaptively learned by the network. Note that α_{ij}^l, β_{ij}^l and γ_{ij}^l can be simple scalar variables, which are shared across all the channels. We force $\alpha_{ij}^l + \beta_{ij}^l + \gamma_{ij}^l = 1$ and $\alpha_{ij}^l, \beta_{ij}^l, \gamma_{ij}^l \in [0, 1]$, and define

$$\alpha_{ij}^{l} = \frac{e^{\lambda_{\alpha_{ij}}^{l}}}{e^{\lambda_{\alpha_{ij}}^{l}} + e^{\lambda_{\beta_{ij}}^{l}} + e^{\lambda_{\gamma_{ij}}^{l}}}. \tag{5}$$

Here α_{ij}^{l}, β_{ij}^{l} and γ_{ij}^{l} are defined by using the softmax function with $\lambda_{\alpha_{ij}}^{l}$, $\lambda_{\beta_{ij}}^{l}$ and $\lambda_{\gamma_{ij}}^{l}$ as control parameters respectively. We use 1×1 convolution layers to compute the weight scalar maps λ_{α}^{l}, λ_{β}^{l} and λ_{γ}^{l} from $\mathbf{x}^{1\to l}$, $\mathbf{x}^{2\to l}$ and $\mathbf{x}^{3\to l}$ respectively, and they can thus be learned through standard back-propagation.

With this method, the features at all the levels are adaptively aggregated at each scale. The outputs $\{\mathbf{y}^1, \mathbf{y}^2, \mathbf{y}^3\}$ are used for object detection following the same pipeline of YOLOv3.

Consistency Property In this section, we analyze the consistency property of the proposed ASFF approach and the other alternatives of feature fusion. Without loss of generality, we focus on the gradient at a certain position (i, j) of the unresized feature maps at level 1 $\mathbf{x}^1$ in YOLOv3. Following the chain rule, the gradient is computed as:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}_{ij}^{1}} = \frac{\partial \mathbf{y}_{ij}^{1}}{\partial \mathbf{x}_{ij}^{1}} \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{y}_{ij}^{1}} + \frac{\partial \mathbf{x}_{ij}^{1\to 2}}{\partial \mathbf{x}_{ij}^{1}} \cdot \frac{\partial \mathbf{y}_{ij}^{2}}{\partial \mathbf{x}_{ij}^{1\to 2}} \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{y}_{ij}^{2}} + \frac{\partial \mathbf{x}_{ij}^{1\to 3}}{\partial \mathbf{x}_{ij}^{1}} \cdot \frac{\partial \mathbf{y}_{ij}^{3}}{\partial \mathbf{x}_{ij}^{1\to 3}} \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{y}_{ij}^{3}} \tag{6}$$

It is worth to note that feature resizing usually uses interpolation for up-sampling and pooling for down-sampling. We thus assume that $\frac{\partial \mathbf{x}_{ij}^{1\to l}}{\partial \mathbf{x}_{ij}^{1}} \approx 1$ for simplicity. Then Eq. (6) can be written as:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}_{ij}^{1}} = \frac{\partial \mathbf{y}_{ij}^{1}}{\partial \mathbf{x}_{ij}^{1}} \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{y}_{ij}^{1}} + \frac{\partial \mathbf{y}_{ij}^{2}}{\partial \mathbf{x}_{ij}^{1\to 2}} \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{y}_{ij}^{2}} + \frac{\partial \mathbf{y}_{ij}^{3}}{\partial \mathbf{x}_{ij}^{1\to 3}} \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{y}_{ij}^{3}} \tag{7}$$

For the two common fusion operations used in RetinaNet [**?**], YOLOv3 [**?**] and other pyramidal feature based detectors (*i.e.* element-wise sum and concatenation), we can further simplify the equation to the following with $\frac{\partial \mathbf{y}_{ij}^{1}}{\partial \mathbf{x}_{ij}^{1}} = 1$ and $\frac{\partial \mathbf{y}_{ij}^{2}}{\partial \mathbf{x}_{ij}^{1\to 2}} = 1$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}_{ij}^{1}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}_{ij}^{1}} + \frac{\partial \mathcal{L}}{\partial \mathbf{y}_{ij}^{2}} + \frac{\partial \mathcal{L}}{\partial \mathbf{y}_{ij}^{3}} \tag{8}$$

Suppose position (i, j) at level 1 is assigned as the center of an object according to a certain scale matching mechanism and $\frac{\partial \mathcal{L}}{\partial \mathbf{y}_{ij}^{1}}$ is the gradient from the positive sample. As the corresponding positions are viewed as background in the other levels, $\frac{\partial \mathcal{L}}{\partial \mathbf{y}_{ij}^{2}}$ and $\frac{\partial \mathcal{L}}{\partial \mathbf{y}_{ij}^{3}}$ are the gradients from negative samples. This inconsistency disturbs the gradient of $\frac{\partial \mathcal{L}}{\partial \mathbf{x}_{ij}^{1}}$ and downgrades the training efficiency of the original feature maps $\mathbf{x}^1$.

One typical way to deal with this problem is to set the corresponding positions of the other levels as ignore regions (*i.e.* $\frac{\partial \mathcal{L}}{\partial \mathbf{y}_{ij}^2} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}_{ij}^3} = 0$). However, although the conflict in $\mathbf{x}_{ij}^1$ is eliminated, the relaxation in $\mathbf{y}_{ij}^2$ and $\mathbf{y}_{ij}^3$ tends to cause more inferior predictions as false positives at the suboptimal levels.

For ASFF, it is straightforward to calculate the gradient from Eq. (4) and Eq. (7) as follows:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}_{ij}^1} = \alpha_{ij}^1 \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{y}_{ij}^1} + \alpha_{ij}^2 \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{y}_{ij}^2} + \alpha_{ij}^3 \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{y}_{ij}^3}, \tag{9}$$

where $\alpha_{ij}^1, \alpha_{ij}^2, \alpha_{ij}^3 \in [0, 1]$. With these three coefficients, the inconsistency of gradient can be harmonized if $\alpha_{ij}^2 \to 0$ and $\alpha_{ij}^3 \to 0$. Since the fusion parameters can be learned by the standard back-propagation algorithm, a well-tuned training process can yield such effective coefficients. Meanwhile, the supervision information of the background in $\frac{\partial \mathcal{L}}{\partial \mathbf{y}_{ij}^2}$ and $\frac{\partial \mathcal{L}}{\partial \mathbf{y}_{ij}^3}$ is kept, avoiding generating more false positives.

A.3 Pseudocode for ResNet-50 enhanced ASFF

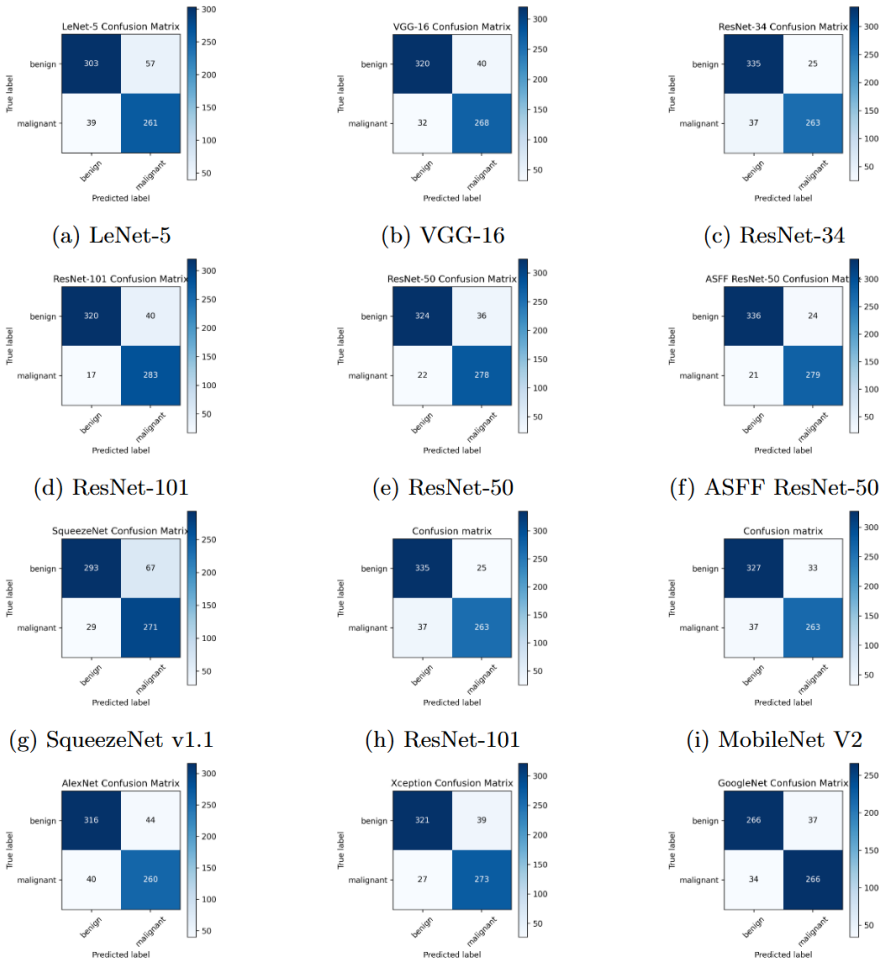
Algorithm 1: ResNet-50 based on ASFF

Input: The image data D_t, classes *c*, epoch *e*, batch size *b*, *input_shape*;
Output: Trained ResNet-50 based on ASFF, saved optimal weights;

```
d ← data_preparation(D_t);
// Data preparation
def Feature_concatenation():
    feature1 ← base_model.get_output('conv4_block6_out');
    feature2 ← base_model.get_output('conv5_block3_out');
    upsampling(feature2, size= feature1_size);
    conv2d(feature2);
    concatenation_output ← feature_Concatenation(feature1, feature2);
    Return concatenation_output.
// Make the size of feature1 and feature2 match, then the two features are concatenated
def Feature_fusion(input_shape, c):
    base_model ← model.application.ResNet50(weights = "imagenet", input_shape =
    input_shape);
    base_model.add(Dense, Upsampling2D, Conv2D);
    Feature_concatenation();
    base_model.add(GlobalAveragePooling2D(concatenation_output));
    base_model.add(input_layer(input = GlobalAveragePooling2D(concatenation_output)));
    base_model.add(Dense, activation = "ReLU");
    base_model.add(output_layer(c, activation = "Softmax"));
    weights ← get_weights(output_layer);
    Fusion_output ← weights.calculate(feature1, feature2);
    Return Fusion_output.
def improved_ResNet50_model (c, input_shape):
    Feature_fusion(input_shape, c);
    model.add(GlobalAveragePooling2D, Dense);
    model.output ← Dense(classes= c, activation = "Sigmoid");
    Return improved_ResNet50_model.
improved_ResNet50_model();
trained_ ResNet_model ← improved_ResNet50_model.train(epoch = e, batch size = b);
//Train the new model
Return Trained ResNet-50 based on ASFF; saved optimal weights.
```

Fig. 7: Pseudocode of the proposed ResNet-50 based on ASFF.

A.4 Confusion Matrices for different models



(a) LeNet-5 (b) VGG-16 (c) ResNet-34

(d) ResNet-101 (e) ResNet-50 (f) ASFF ResNet-50

(g) SqueezeNet v1.1 (h) ResNet-101 (i) MobileNet V2

(j) AlexNet (k) Xception (l) GoogleNet

Fig. 8: Confusion matrices for the validated models.